CAHIER DES CHARGES FONCTIONNEL

ET DESCRIPTION DE STAGE

Stage d'été - Data Science, Edge AI, IoT industriel et IA générative contrôlée

Edge AI explicable et agent IA/RAG pour la détection d'anomalies industrielles sur Raspberry Pi

Résumé : Ce document présente le cahier des charges fonctionnel d'un stage d'été visant à développer un prototype Edge AI explicable pour la détection d'anomalies industrielles à partir de données capteurs publiques ou simulées. Le travail inclut l'entraînement de modèles légers, leur déploiement sur Raspberry Pi, un benchmark resource-aware, une analyse d'explicabilité et un agent IA augmenté par RAG pour générer des explications et des rapports de diagnostic.

Mots clés : Edge AI, Raspberry Pi, IoT industriel, détection d'anomalies, maintenance prédictive, explicabilité, RAG, agent IA, benchmark embarqué, données capteurs.

Table des matières

- 1. Description synthétique du stage
- 2. Contexte du sujet
- 3. Problématique, objectif général et hypothèse scientifique
- 4. Périmètre du stage
- 5. Architecture fonctionnelle attendue
- 6. Cahier des charges fonctionnel détaillé
- 7. Exigences non fonctionnelles
- 8. Technologies recommandées
- 9. Planning détaillé sur huit semaines
- 10. Livrables attendus
- 11. Critères d’évaluation du stage
- 12. Valorisation scientifique attendue
- 13. Risques, limites et stratégies de mitigation
- 14. Annexes opérationnelles pour l’étudiant

1. Description synthétique du stage

Ce stage d'été porte sur le développement d'un prototype expérimental combinant Edge AI, IoT industriel, explicabilité et agent IA augmenté par RAG. L'étudiant travaillera sur des données capteurs publiques, simulées ou hybrides afin de détecter des anomalies industrielles, puis déploiera le modèle retenu sur une Raspberry Pi 4 ou 5. Le stage aboutira à un prototype fonctionnel, un benchmark expérimental, un agent d'explication et un draft d'article scientifique.

Le cœur du travail consiste à construire un pipeline complet : préparation des données, extraction de caractéristiques, comparaison de modèles légers, déploiement embarqué, mesure des contraintes Edge AI, interprétation des décisions et génération automatique d'un rapport de diagnostic à l'aide d'un agent IA/RAG.

Rubrique	Contenu
Intitulé proposé	Edge AI explicable et agent IA/RAG pour la détection d'anomalies industrielles sur Raspberry Pi
Durée	8 semaines
Profil recherché	Étudiant avec bases en Python, Data Science, machine learning et systèmes embarqués
Nature du travail	Prototype expérimental, benchmark embarqué, explicabilité, assistant IA et valorisation scientifique
Résultat principal	Démonstrateur fonctionnel avec modèle IA léger déployé sur Raspberry Pi et agent de diagnostic
Valorisation	Communication scientifique ou article court basé sur le protocole et les résultats expérimentaux

2. Contexte et justification du sujet

Les systèmes industriels modernes génèrent en continu des données issues de capteurs : vibration, température, courant, pression, vitesse, signaux inertiels ou séries temporelles provenant de systèmes IoT. Ces données peuvent aider à détecter des anomalies, des défauts qualité, des dérives de fonctionnement ou des besoins de maintenance avant l'apparition d'une panne critique.

L'approche classique consiste souvent à transmettre les données vers un serveur ou un cloud pour analyse. Cette approche peut devenir limitée dans des contextes industriels où la latence, la confidentialité, la disponibilité du réseau et le coût de transmission sont des contraintes importantes. L'Edge AI propose alors d'exécuter les modèles d'intelligence artificielle à proximité des données, sur des dispositifs comme des passerelles IoT, des ordinateurs industriels ou une Raspberry Pi.

L'utilisation d'une Raspberry Pi 4 ou 5 permet de transformer le stage en démonstrateur réel d'IA embarquée. Le modèle ne sera pas seulement entraîné sur ordinateur : il devra être exécuté localement sur la carte, avec mesure du temps d'inférence, de la mémoire utilisée, de la taille du modèle et de la robustesse en conditions dégradées.

Le stage ajoute également une couche d'agent IA augmenté par RAG. Cet agent ne remplace pas le modèle de détection. Il sert à expliquer les résultats, contextualiser les anomalies, générer un mini-rapport de diagnostic et aider l'étudiant ou l'utilisateur à comprendre les décisions du système.

3. Problématique, objectif général et hypothèse scientifique

3.1. Problématique

Comment concevoir, déployer et expliquer un modèle Edge AI léger sur Raspberry Pi pour détecter des anomalies industrielles à partir de données capteurs publiques ou simulées, tout en utilisant un agent IA/RAG pour produire une explication technique, contextualisée et exploitable pédagogiquement ?

3.2. Objectif général

Développer un prototype expérimental combinant un modèle Edge AI léger, un déploiement réel sur Raspberry Pi, un benchmark précision/latence/mémoire, une analyse d'explicabilité et un agent IA/RAG capable de générer des explications et des rapports de diagnostic.

3.3. Objectifs spécifiques

- Sélectionner un cas industriel réaliste de détection d'anomalies : vibration anormale, défaut mécanique, dérive thermique, anomalie capteur ou défaut de fonctionnement.
- Utiliser un dataset public, simulé ou hybride de données capteurs, avec documentation claire de la source, des variables et des labels. (CWRU Bearing Dataset, NASA C-MAPSS, MIMII Dataset, SWaT / WADI).
- Préparer les données : nettoyage, fenêtrage temporel, normalisation et extraction de caractéristiques.
- Comparer plusieurs modèles IA légers adaptés à l'Edge AI.
- Déployer le modèle retenu sur Raspberry Pi 4 ou 5.
- Mesurer la performance de détection et les contraintes embarquées : latence, mémoire, CPU, taille du modèle et débit d'inférence.
- Ajouter une analyse d'explicabilité : variables importantes, fenêtres critiques, erreurs du modèle, faux positifs et faux négatifs.
- Concevoir un agent IA/RAG pour expliquer les anomalies et générer un rapport de diagnostic.
- Transformer le démonstrateur en cas pédagogique avec énoncé, questions et grille d'évaluation.
- Produire un rapport final.

3.4. Hypothèse scientifique

Un modèle IA léger, déployé sur Raspberry Pi et associé à un protocole de benchmark resource-aware, peut détecter des anomalies industrielles avec un compromis acceptable entre performance, latence, mémoire et explicabilité. L'ajout d'un agent IA/RAG permet d'améliorer l'interprétation des résultats, la traçabilité des explications et la transformation pédagogique du cas.

4. Périmètre du stage

4.1. Ce qui est inclus

Axe	Travail attendu
Données	Utilisation d'un dataset public, simulé ou hybride ; documentation et préparation des données.
Prétraitement	Nettoyage, segmentation temporelle, extraction de caractéristiques et séparation train/test.
Modélisation IA	Comparaison de modèles légers pour la détection d'anomalies.
Implémentation Edge	Déploiement réel du modèle sur Raspberry Pi 4 ou 5.
Benchmark	Mesure de précision, rappel, F1-score, AUC, latence, taille modèle, RAM et CPU.
Explicabilité	Analyse des variables importantes, fenêtres critiques et erreurs.
Agent IA/RAG	Base documentaire, recherche d'information, génération d'explication et rapport de diagnostic.
Pédagogie	Transformation en activité pédagogique structurée.
Valorisation scientifique	Figures, tableaux, protocole expérimental et draft d'article.

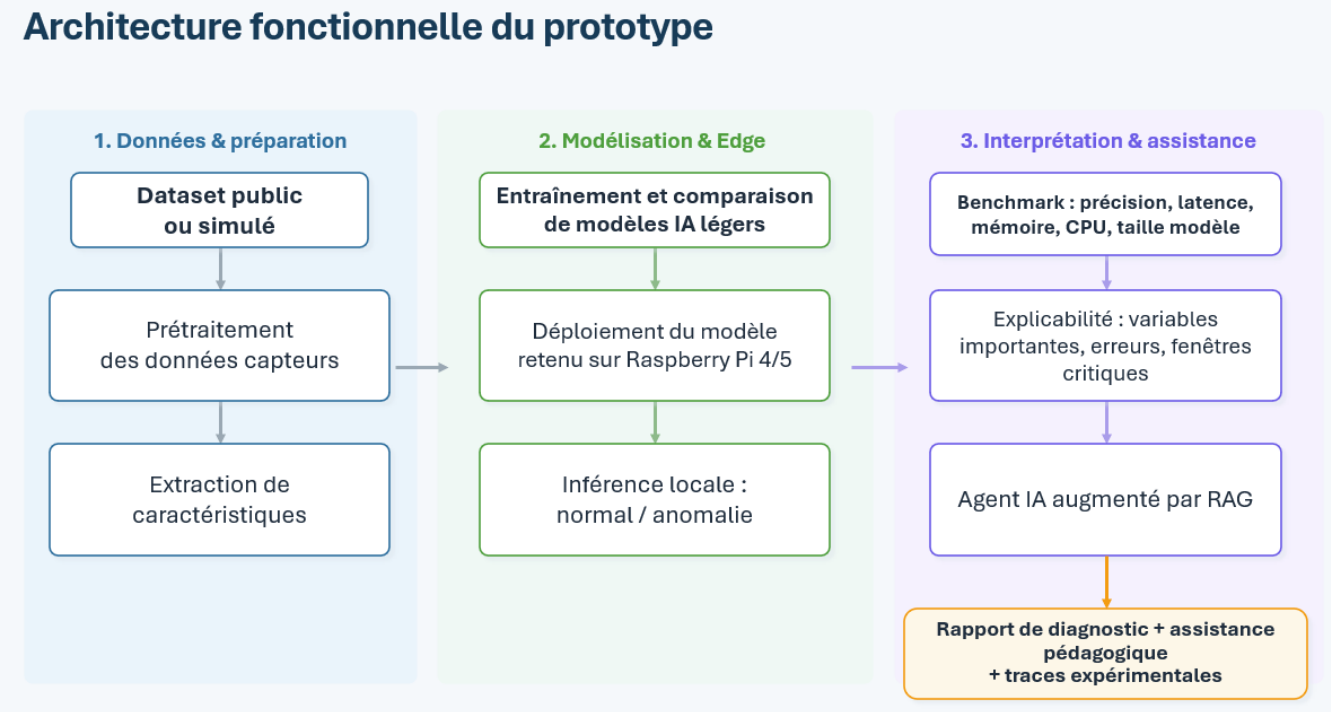
4.2. Ce qui est exclu

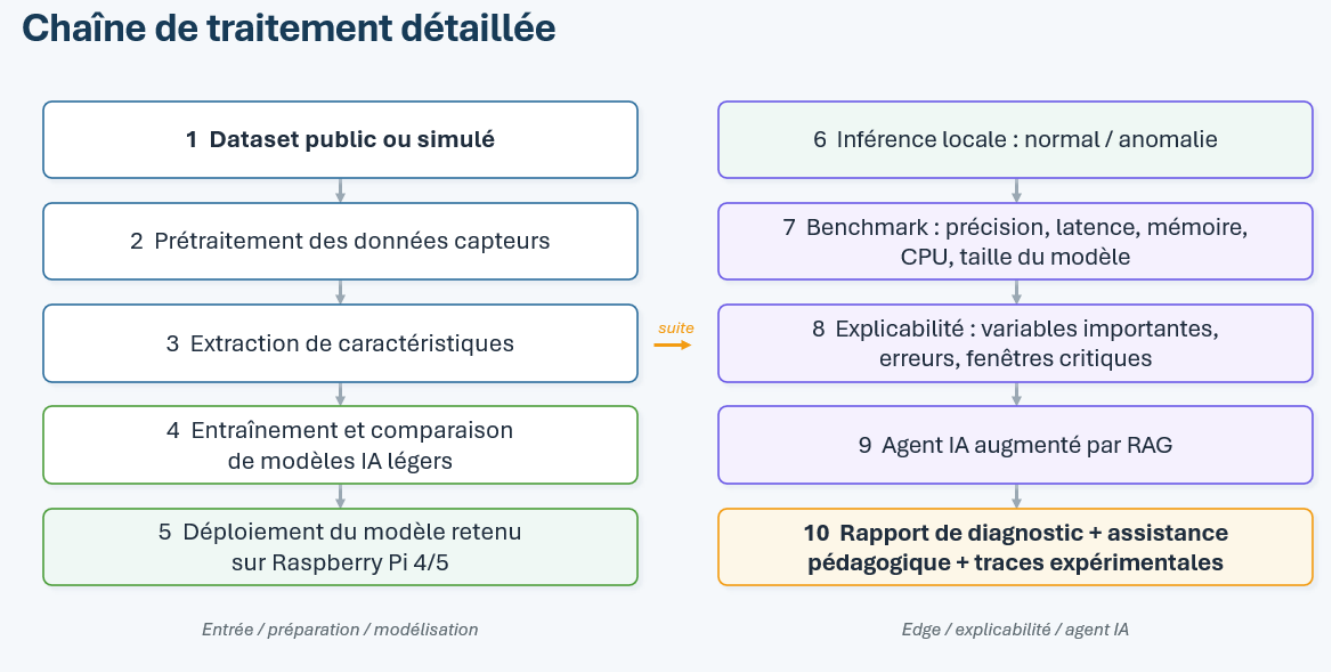
Élément exclu	Justification
Acquisition capteurs temps réel	Le stage ne dépend pas d'un banc de capteurs physique. Le flux capteur peut être simulé à partir d'un dataset.
Déploiement industriel complet	La durée du stage ne permet pas de valider le système sur une chaîne industrielle réelle.

Multiplication des cas industriels	Le stage doit rester centré sur un seul cas principal pour garantir la rigueur expérimentale.
LLM local lourd sur Raspberry Pi	Le modèle de langage peut être exécuté sur PC ou via API. La Raspberry Pi est réservée à l'inférence Edge AI.
Système logiciel complexe	L'objectif est un prototype expérimental robuste, pas une plateforme logicielle complète.

5. Architecture fonctionnelle attendue

Le système attendu doit être organisé autour d'une chaîne complète allant des données jusqu'au diagnostic explicable. La Raspberry Pi joue le rôle de passerelle Edge exécutant le modèle de détection. L'agent IA/RAG intervient après la détection pour produire une explication contextualisée et un rapport de diagnostic.





6. Cahier des charges fonctionnel détaillé

6.1. Module 1 - Gestion et préparation des données

Le système doit permettre de charger, nettoyer, structurer et documenter un dataset de données capteurs. Comme le stage ne repose pas sur un banc de capteurs physique, la stratégie de données doit garantir la faisabilité dès la première semaine.

Type de données	Description
Dataset public	Données de vibration, moteur, roulement, machine ou process industriel. Exemples possibles : CWRU Bearing Dataset, NASA C-MAPSS, MIMII Dataset.
Dataset simulé	Signaux générés pour représenter un fonctionnement normal et plusieurs anomalies contrôlées.
Dataset hybride	Dataset public enrichi avec bruit, dérive, perte de données, pics ou capteur bloqué.
Dataset pédagogique réduit	Version simplifiée et documentée pour l'activité étudiante et le test de l'agent IA/RAG.

Code	Fonctionnalité attendue
F1.1	Charger les données depuis un fichier CSV, TXT, MAT, NPZ ou format équivalent.
F1.2	Nettoyer les valeurs manquantes, doublons et valeurs aberrantes.
F1.3	Segmenter les signaux temporels en fenêtres exploitables.

F1.4	Associer un label à chaque fenêtre : normal, anomalie ou type d'anomalie si disponible.
F1.5	Documenter le dataset : source, variables, fréquence d'échantillonnage, labels, taille, limites.
F1.6	Prévoir un script reproductible permettant de reconstruire le dataset préparé.
F1.7	Créer une version réduite du dataset pour démonstration et pédagogie.

6.2. Module 2 - Prétraitement, fenêtrage et extraction de caractéristiques

Les signaux bruts doivent être transformés en données exploitables par les modèles. Le stagiaire devra justifier les choix de taille de fenêtre, recouvrement, normalisation et features extraites.

Famille de caractéristiques	Exemples
Temporelles	Moyenne, écart-type, variance, RMS, minimum, maximum, amplitude, énergie.
Statistiques	Skewness, kurtosis, facteur de crête, facteur d'impulsion.
Fréquentielles	Énergie spectrale, fréquence dominante, bandes de fréquence simples.
Qualité du signal	Taux de valeurs manquantes, bruit, dérive, pics, stabilité.
Métadonnées	Identifiant de fenêtre, classe, scénario, niveau de bruit, source de données.

Code	Fonctionnalité attendue
F2.1	Normaliser ou standardiser les signaux selon une stratégie documentée.
F2.2	Découper les signaux en fenêtres temporelles avec ou sans recouvrement.
F2.3	Extraire des caractéristiques pour chaque fenêtre.
F2.4	Séparer les données en train, validation et test sans fuite de données.
F2.5	Produire un notebook d'analyse exploratoire avec figures lisibles.
F2.6	Visualiser au moins trois exemples de comportement normal et trois exemples d'anomalie.
F2.7	Sauvegarder un fichier final prêt pour l'entraînement des modèles.

6.3. Module 3 - Modélisation IA légère

Le stagiaire devra comparer plusieurs modèles simples et légers. La priorité n'est pas de produire le modèle le plus complexe, mais d'obtenir un compromis robuste entre performance de détection, rapidité, taille et explicabilité. Exemple :

Modèle	Rôle dans le benchmark
Isolation Forest	Baseline utile pour détection d'anomalies non supervisée ou semi-supervisée.
Random Forest léger	Modèle robuste, simple à entraîner et partiellement interprétable.
XGBoost léger	Modèle performant pouvant rester compatible avec une inférence rapide.
Autoencodeur compact	Option intéressante pour apprendre le fonctionnement normal et détecter les écarts.
1D-CNN léger	Option avancée si les signaux temporels bruts sont utilisés directement.

Code	Fonctionnalité attendue
F3.1	Entraîner un modèle de référence sans optimisation Edge.
F3.2	Entraîner au moins deux modèles légers supplémentaires.
F3.3	Évaluer accuracy, precision, recall, F1-score, AUC et matrice de confusion.
F3.4	Analyser les faux positifs et les faux négatifs.
F3.5	Sauvegarder les modèles dans un format réutilisable : joblib, pickle, ONNX ou TFLite selon le cas (quantification et optimisation)
F3.6	Sélectionner le modèle candidat pour Raspberry Pi à partir d'un tableau de décision.
F3.7	Justifier le choix final par des critères techniques et non uniquement par la précision.

6.4. Module 4 - Déploiement sur Raspberry Pi 4 ou 5

Le modèle retenu doit être exécuté localement sur Raspberry Pi. Le flux capteur peut être simulé en lisant successivement des fenêtres depuis un fichier, avec un délai configurable pour imiter un flux IoT.

Code	Fonctionnalité attendue
F4.1	Installer l'environnement Python sur Raspberry Pi : bibliothèques de base, dépendances ML, outils de mesure.

F4.2	Transférer le modèle entraîné et les scripts d'inférence vers la Raspberry Pi.
F4.3	Charger une fenêtre de données et produire une décision localement.
F4.4	Simuler un flux capteur en envoyant les fenêtres une par une.
F4.5	Afficher la décision : normal, anomalie ou type d'anomalie.
F4.6	Enregistrer les résultats dans un fichier log : horodatage, fenêtre, prédiction, score, latence.
F4.7	Mesurer le temps d'inférence moyen, médian, maximal et l'écart-type.
F4.8	Mesurer la mémoire RAM utilisée et la charge CPU approximative.
F4.9	Prévoir un mode démonstration simple en ligne de commande ou interface légère.

6.5. Module 5 - Benchmark Edge AI resource-aware

Le benchmark doit comparer les modèles selon deux familles de critères : la qualité de détection et les contraintes embarquées. Le résultat le plus important du stage sera le compromis entre performance, latence, mémoire, taille et robustesse.

Critère	Mesure attendue
Performance de détection	Accuracy, precision, recall, F1-score, AUC, matrice de confusion.
Latence	Temps d'inférence moyen, médian, maximal, écart-type sur Raspberry Pi.
Mémoire	RAM utilisée pendant le chargement du modèle et pendant l'inférence.
Taille	Taille du modèle sauvegardé sur disque.
Débit	Nombre de fenêtres traitées par seconde.
CPU	Charge processeur approximative pendant l'inférence.
Robustesse	Performance sous bruit, données manquantes, dérive progressive ou pics.
Simplicité	Nombre de features, paramètres du modèle, facilité de déploiement.

Code	Fonctionnalité attendue
F5.1	Construire un tableau comparatif des modèles selon les métriques de détection.
F5.2	Construire un tableau comparatif des modèles selon les métriques Edge.
F5.3	Tester au moins trois scénarios de robustesse : bruit, dérive, perte de données ou pics.

F5.4	Identifier le modèle le plus équilibré, pas seulement le plus précis.
F5.5	Produire au moins trois figures scientifiques : matrice de confusion, courbe de performance, comparaison latence/mémoire.
F5.6	Rédiger une discussion sur les compromis observés.

6.6. Module 6 - Explicabilité et analyse d'erreurs

L'explicabilité doit aider à comprendre pourquoi le modèle détecte une anomalie. Elle doit être reliée aux variables capteurs, aux caractéristiques extraites et aux exemples de fenêtres normales ou anormales.

Axe XAI	Travail attendu
Importance des variables	Identifier les caractéristiques qui influencent le plus la décision.
Analyse locale	Expliquer quelques fenêtres précises détectées comme anomalies.
Fenêtres critiques	Visualiser les zones temporelles associées à la détection.
Erreurs	Étudier les faux positifs et faux négatifs.
Interprétation industrielle	Relier les signaux et features à une hypothèse de défaut ou de dérive.
Limites	Indiquer ce que l'explication ne permet pas de conclure.

Code	Fonctionnalité attendue
F6.1	Calculer ou estimer l'importance des features.
F6.2	Produire des graphiques lisibles sur les features importantes.
F6.3	Analyser au moins cinq anomalies correctement détectées.
F6.4	Analyser au moins trois faux positifs ou faux négatifs si disponibles.
F6.5	Rédiger une interprétation technique compréhensible.
F6.6	Préparer des résultats XAI qui pourront être utilisés par l'agent IA/RAG.

6.7. Module 7 - Agent IA augmenté par RAG

L'agent IA/RAG est une couche d'assistance. Il ne réalise pas la détection d'anomalies. Il reçoit les résultats du modèle, les métriques, les informations d'explicabilité et interroge une base documentaire contrôlée pour produire une réponse claire, traçable et contextualisée.

Rôle	Description
Explication d'anomalie	Traduire une décision du modèle en explication technique lisible.

Diagnostic structuré	Générer une fiche avec fenêtre, modèle, score, variables importantes, interprétation et limite.
Assistance pédagogique	Répondre aux questions sur les données, les métriques, la latence, la mémoire ou l'explicabilité.
Réduction des hallucinations	S'appuyer sur une base documentaire limitée et vérifiée.
Traçabilité	Indiquer les documents ou sections utilisés pour générer la réponse.
Évaluation	Tester l'agent sur une liste de questions et vérifier la pertinence des réponses.

Document	Contenu attendu dans la base RAG
D1	Description du dataset : source, variables, classes, limites.
D2	Fiche technique sur le cas étudié : défaut mécanique, vibration, dérive ou anomalie capteur.
D3	Description des features : RMS, kurtosis, énergie, fréquence dominante, etc.
D4	Résultats du benchmark : métriques, latence, mémoire, taille modèle.
D5	Résultats d'explicabilité : variables importantes, exemples d'erreurs.
D6	Mini-cours : Edge AI, Raspberry Pi, F1-score, recall, precision, AUC.
D7	Énoncé pédagogique du cas et questions progressives.
D8	Grille d'évaluation des réponses étudiantes ou des diagnostics.

Code	Fonctionnalité attendue
F7.1	Construire une base documentaire courte, propre et contrôlée.
F7.2	Segmenter les documents en passages exploitables.
F7.3	Indexer les passages dans une base vectorielle : FAISS, ChromaDB ou équivalent.
F7.4	Permettre une recherche documentaire à partir d'une question.
F7.5	Générer une explication d'anomalie à partir des résultats du modèle.
F7.6	Générer un mini-rapport de diagnostic structuré.
F7.7	Afficher les sources utilisées par l'agent ou au minimum les documents consultés.
F7.8	Évaluer l'agent sur 10 à 20 questions ou cas de diagnostic.
F7.9	Identifier les limites : réponse incomplète, contradiction, hallucination, manque de contexte.

Exemple de sortie attendue de l'agent IA/RAG

```
Diagnostic généré par l'agent IA

Fenêtre analysée : 128
```

```
Décision du modèle : anomalie
Modèle utilisé : Random Forest léger
Latence sur Raspberry Pi : 7,8 ms
Variables importantes : RMS, kurtosis, énergie spectrale

Interprétation :
L'anomalie est probablement liée à une variation anormale du signal vibratoire. L'augmentation
de la valeur RMS et de la kurtosis indique un comportement non régulier par rapport au
fonctionnement normal.

Limite :
Le diagnostic est basé sur un dataset public ou simulé. Une validation sur des données
industrielles réelles sera nécessaire avant toute décision opérationnelle.
```

6.8. Module 8 - Transformation pédagogique du démonstrateur

Le démonstrateur doit être transformé en activité pédagogique exploitable par des étudiants ingénieurs. Cette transformation est importante pour donner une valeur académique au stage et préparer une valorisation scientifique autour de l'apprentissage par problèmes industriels.

Élément	Contenu attendu
Contexte industriel	Scénario réaliste expliquant la machine, les capteurs, l'anomalie et l'objectif de diagnostic.
Données fournies	Dataset réduit, documenté et adapté à une activité étudiante.
Questions progressives	Compréhension, visualisation, features, modèle, benchmark, explicabilité, diagnostic.
Compétences visées	Data Science, IoT industriel, Edge AI, diagnostic, argumentation et communication technique.
Livrables étudiants	Notebook, mini-rapport, analyse des résultats, décision technique.
Grille d'évaluation	Critères détaillés avec niveaux de performance.
Agent IA	Utilisé comme assistant de compréhension et non comme outil donnant directement la solution.

Code	Fonctionnalité attendue
F8.1	Rédiger un énoncé pédagogique clair.
F8.2	Préparer les données étudiants avec documentation.
F8.3	Construire des questions progressives.
F8.4	Définir les compétences visées.
F8.5	Préparer une grille d'évaluation.
F8.6	Prévoir une activité utilisant l'agent IA/RAG pour expliquer les résultats.
F8.7	Rédiger un court guide enseignant indiquant comment exploiter le cas.

7. Exigences non fonctionnelles

Exigence	Description
Reproductibilité	Le code doit permettre de reproduire le dataset préparé, l'entraînement, le benchmark et les figures.
Simplicité	Le prototype doit rester faisable en huit semaines et éviter les architectures inutilement complexes.
Traçabilité	Chaque expérimentation doit être identifiée : date, dataset, modèle, paramètres, métriques et environnement.
Confidentialité	Aucune donnée confidentielle ne doit être utilisée. Les données doivent être publiques, simulées ou anonymisées.
Compatibilité Edge	Le modèle retenu doit fonctionner sur Raspberry Pi 4 ou 5.
Robustesse	Le modèle doit être testé sur au moins trois perturbations : bruit, perte de données, dérive ou pics.
Explicabilité	Les décisions doivent être interprétables et reliées aux caractéristiques des signaux.
Traçabilité RAG	Les réponses de l'agent doivent s'appuyer sur une base documentaire contrôlée.
Qualité scientifique	Les résultats doivent pouvoir être transformés en figures, tableaux et discussion d'article.
Qualité pédagogique	Le cas final doit être compréhensible et exploitable par des étudiants ingénieurs.

8. Planning détaillé sur huit semaines

Semaine	Travail demandé	Livrable
S1	Cadrage scientifique, choix du dataset, recherche bibliographique, installation environnement PC et Raspberry Pi.	Note de cadrage, environnement prêt, stratégie de données validée.
S2	Préparation des données : import, nettoyage, visualisation, fenêtrage, documentation.	Dataset préparé, notebook d'analyse exploratoire, premières figures.
S3	Extraction de features, entraînement du modèle de référence, premières métriques.	Modèle baseline, tableau de métriques, analyse préliminaire.
S4	Comparaison de modèles légers, réduction éventuelle de features, sélection du modèle candidat.	Tableau comparatif des modèles et justification du choix.

S5	Déploiement sur Raspberry Pi, simulation du flux capteur, mesure latence, mémoire, CPU et taille modèle.	Benchmark Edge AI complet.
S6	Explicabilité, analyse des erreurs, figures scientifiques et interprétation technique.	Rapport XAI, figures, analyse des faux positifs/faux négatifs.
S7	Développement de l'agent IA/RAG, base documentaire, génération de diagnostic, transformation pédagogique.	Prototype agent, mini-rapport généré, énoncé pédagogique et grille.
S8	Finalisation du rapport, préparation du draft d'article, présentation finale et dossier de livrables.	Rapport final, présentation, draft d'article et code organisé.

10. Livrables attendus

Code	Livrable
L1	Note de cadrage scientifique
L2	Dataset documenté et script de préparation
L3	Notebook d'analyse exploratoire
L4	Scripts de prétraitement et d'extraction de caractéristiques
L5	Modèles IA entraînés et sauvegardés
L6	Modèle déployé sur Raspberry Pi
L7	Scripts d'inférence locale et logs Raspberry Pi
L8	Tableau de benchmark précision/latence/mémoire/CPU/taille
L9	Rapport d'explicabilité et analyse d'erreurs
L10	Prototype d'agent IA/RAG
L11	Mini-rapport de diagnostic généré automatiquement
L12	Cas pédagogique complet
L13	Grille d'évaluation des compétences
L14	Rapport final de stage
L15	Draft d'article scientifique
L16	Présentation finale

L17	Dossier de code propre avec README et instructions d'exécution

11. Annexes opérationnelles pour l'étudiant

11.1. Checklist de démarrage

- Créer un dossier de travail structuré : data, notebooks, src, models, raspberry, rag, reports, figures.
- Installer l'environnement Python sur PC et vérifier les versions des bibliothèques.
- Installer Raspberry Pi OS et tester Python, réseau, accès SSH et copie de fichiers.
- Choisir le dataset principal et rédiger une fiche de description.
- Définir la tâche : normal/anomalie ou classification de défauts.
- Définir les métriques de détection et les métriques Edge.
- Créer un fichier de journal d'expériences dès la première semaine.

11.2. Structure recommandée du dépôt de code

```
project/
  data/
    raw/
    processed/
    pedagogical_subset/
  notebooks/
    01_exploration.ipynb
    02_features_and_models.ipynb
  src/
    data_preparation.py
    feature_extraction.py
    train_models.py
    evaluate_models.py
    explainability.py
  raspberry/
    inference_edge.py
    benchmark_edge.py
    logs/
  rag_agent/
    documents/
    build_index.py
    ask_agent.py
    generate_diagnostic.py
  models/
  figures/
  reports/
  README.md
```

11.3. Modèle de fiche dataset

Champ	À compléter par l'étudiant
Nom du dataset	
Source	
Type de signaux	
Fréquence d'échantillonnage	
Variables disponibles	
Classes ou labels	
Taille totale	
Méthode de fenêtrage	
Méthode de séparation train/test	
Limites connues	

12. Résultat minimal attendu

Le stage sera considéré comme réussi si l'étudiant produit au minimum un dataset documenté, un modèle IA léger fonctionnel, une implémentation réelle sur Raspberry Pi, un benchmark précision/latence/mémoire, une analyse explicable des anomalies, un prototype simple d'agent IA/RAG, un cas pédagogique, un rapport final et un draft d'article scientifique.

Fin du cahier des charges